

5.6 Consider the bank database of Figure 5.21. Let us define a view branch_cust as follows:

```
create view branch_cust as
select account.branch_name, depositor.customer_name
from depositor
join account
  on depositor.account_number = account.account_number;
```

Suppose that the view is materialized; that is, the view is computed and stored. Write triggers to maintain the view, that is, to keep it up-to-date on insertions to depositor or account. It is not necessary to handle deletions or updates. Note that, for simplicity, we have not required the elimination of duplicates.

branch (branch_name, branch_city, assets)
customer (customer_name, customer_street, customer_city)
loan (loan_number, branch_name, amount)
borrower (customer_name, loan_number)
account (account_number, branch_name, balance)
depositor (customer_name, account_number)

Figure 5.21 Banking database for Exercise 5.6.

```
create trigger depositor_check
after insert on depositor
referencing new row as nrow
for each row
begin
  insert into branch_cust (branch_name, customer_name)
  select a.branch_name, nrow.customer_name
  from account as a
  where a.account_number = nrow.account_number;
end;

create trigger account_check
after insert on account
referencing new row as nrow
for each row
begin
  insert into branch_cust (branch_name, customer_name)
  select nrow.branch_name, d.customer_name
  from depositor as d
  where d.account_number = nrow.account_number;
end;
```

5.15 Consider an employee database with two relations

employee (employee_name, street, city)
works (employee_name, company_name, salary)

where the primary keys are underlined. Write a function avg salary that takes a company name as an argument and finds the average salary of employees at that company. Then, write an SQL statement, using that function, to find companies whose employees earn a higher salary, on average, than the average salary at “First Bank”.

```
-- 接收公司名称, 返回该公司员工的平均薪资
create function avg_salary(p_company_name varchar(20))
returns numeric(10,2)
begin
    declare v_avg_salary numeric(10,2);

    select avg(w.salary)
    into v_avg_salary
    from works as w
    where w.company_name = p_company_name;

    return v_avg_salary;
end;

-- 查询平均薪资高于 First Bank 的公司
select distinct w.company_name
from works as w
where avg_salary(w.company_name) > avg_salary('First Bank');
```

另：写一个嵌入 SQL/ODBC 程序或 Stored Procedure，保存一位同学的一门选课信息，需检查不能有冲突的上课时间；所有先修课必须通过；教室容量必须够。如果以上条件不满足则失败。



3 / 5

```
from takes as t
where t.course_id = p_course_id
    and t.sec_id = p_sec_id
    and t.semester = p_semester
    and t.year = p_year;

if v_enrolled_num >= v_room_capacity then
    signal sqlstate '45000'
        set message_text = 'error: classroom capacity is full';
end if;

-- 2. 检查先修课是否全部通过
select count(*)
into v_unpassed_prereq
from prereq as p
where p.course_id = p_course_id
    and not exists (
        select 1
        from takes as t
        where t.id = p_student_id
            and t.course_id = p.prereq_id
            and t.grade is not null
    );

if v_unpassed_prereq > 0 then
    signal sqlstate '45000'
        set message_text = 'error: prerequisites not passed';
end if;

-- 3. 检查上课时间是否冲突
select count(*)
into v_time_conflict
from takes as t
join section as s_old
    on t.course_id = s_old.course_id
    and t.sec_id = s_old.sec_id
    and t.semester = s_old.semester
    and t.year = s_old.year
join time_slot as ts_old
    on s_old.time_slot_id = ts_old.time_slot_id
join section as s_new
    on s_new.course_id = p_course_id
    and s_new.sec_id = p_sec_id
    and s_new.semester = p_semester
    and s_new.year = p_year
join time_slot as ts_new
    on s_new.time_slot_id = ts_new.time_slot_id
where t.id = p_student_id
    and ts_old.day = ts_new.day
    and ts_old.start_time < ts_new.end_time
    and ts_old.end_time > ts_new.start_time;

if v_time_conflict > 0 then
    signal sqlstate '45000'
```

```
        set message_text = 'error: class time conflict';
    end if;

    insert into takes (id, course_id, sec_id, semester, year, grade)
    values (p_student_id, p_course_id, p_sec_id, p_semester, p_year, p_grade);

    select 'success: course enrollment completed' as result;
end //

delimiter ;
```